

ANÁLISE DE ALGORITMOS DE ORDENAÇÃO

Critério de contagem utilizado

- Comparação: toda vez que dois valores são comparados para decidir a ordenação (ex.: $v[j] > v[j+1]$).
- Bubble Sort - Troca: cada swap efetivamente realizado quando um par está fora de ordem.
- Insertion Sort - Movimentação: cada deslocamento de elemento uma posição à frente durante a inserção (não conta a colocação final da chave).
- Selection Sort - Troca: uma troca por passagem, apenas quando o menor elemento encontrado não está na posição já correta.
- Quick Sort - Movimentação: elementos reposicionados durante o particionamento (implementado com partição por listas, comparando cada elemento ao pivô).

Resultados

Tam.	Bubble-Comp	Bubble-Troc	Insertion-Comp	Insertion-Mov	Selection-Comp	Selection-Troc	Quick-Comp	Quick-Mov
10	45	26	33	26	45	6	27	33
20	187	114	131	114	190	16	58	70
1.000	499.329	241.624	242.617	241.624	499.500	990	11.408	12.073

Análise dos resultados

a) Menor número de comparações com 10 elementos? Quick Sort (27), seguido de Insertion Sort (33).

b) Menos trocas/movimentações? Selection Sort (6), pois só troca uma vez por passagem.

c) Comportamento semelhante com 20 elementos? Sim, a mesma ordem de eficiência se mantém: Quick Sort < Insertion Sort < Bubble Sort $\approx$ Selection Sort.

d) O que ocorre com 1.000 elementos? Bubble e Selection Sort explodem para ~500 mil comparações; Insertion Sort cresce de forma parecida (~242 mil); Quick Sort permanece muito abaixo (~11 mil), evidenciando a diferença de complexidade.

e) Bubble, Insertion e Selection são todos $O(n^2)$: fizeram o mesmo número de operações? Não. Embora os três sejam $O(n^2)$ no caso médio/pior caso, Bubble e Selection tiveram números de comparações praticamente idênticos (499.329 e 499.500, ambos próximos de $n^2/2$), mas o Insertion Sort teve menos da metade das comparações (242.617), pois ele para de comparar assim que encontra a posição correta, sem precisar varrer o restante do vetor. Ou seja, a mesma ordem de grandeza ($O(n^2)$), mas com constantes diferentes.

f) Maior crescimento no número de operações? Bubble Sort e Selection Sort, que crescem de forma quadrática ($O(n^2)$) e dispararam ao passar de 20 para 1.000 elementos.

g) Como o Quick Sort se diferenciou? Cresceu de forma muito mais lenta ($O(n \log n)$): com 1.000 elementos, usou apenas ~11 mil comparações contra ~500 mil dos algoritmos $O(n^2)$, uma diferença de mais de 40 vezes.

h) Resultados coerentes com a teoria? Sim. O crescimento quadrático de Bubble/Insertion/Selection e o crescimento quase linear ($n \log n$) do Quick Sort observados nos experimentos correspondem exatamente ao esperado teoricamente.

i) Qual algoritmo escolher para milhares de pedidos? Quick Sort, pois apresentou o menor número de operações em todos os tamanhos testados e a menor taxa de crescimento, é o mais escalável para grandes volumes de dados.

Desafio adicional: Aleatório x Ordenado x Invertido ($n = 1.000$)

Caso	Bubble(c,t)	Insertion(c,m)	Selection(c,t)	Quick(c,m)
Aleatório	499.329 / 241.624	242.617 / 241.624	499.500 / 990	11.408 / 12.073
Ordenado	999 / 0	999 / 0	499.500 / 0	499.500 / 500.499

Caso	Bubble(c,t)	Insertion(c,m)	Selection(c,t)	Quick(c,m)
Invertido	499.500 / 499.494	499.500 / 499.494	499.500 / 502	497.115 / 498.108

Análise: a organização inicial dos dados NÃO afeta todos os algoritmos da mesma forma.

- Bubble e Insertion Sort: melhor caso é o vetor já ordenado (apenas 999 comparações, $O(n)$, pois param assim que percebem que não há mais trocas/inversões) e pior caso é o vetor invertido (499.500 comparações, $O(n^2)$).
- Selection Sort: praticamente não é afetado pela organização inicial, sempre faz ~499.500 comparações, pois sempre varre todo o restante do vetor em busca do menor elemento, independentemente da ordem.
- Quick Sort: é o mais sensível de todos, no vetor aleatório teve ótimo desempenho (~11 mil comparações), mas no vetor já ordenado (usando o último elemento como pivô) caiu no seu pior caso, com 499.500 comparações, igual aos algoritmos $O(n^2)$.

Conclusão: a organização inicial dos dados interfere de maneira diferente em cada algoritmo. Bubble e Insertion Sort se beneficiam de dados já ordenados; Selection Sort é indiferente à ordem inicial; e o Quick Sort, apesar de ser o mais eficiente em média, pode ter seu desempenho degradado ao pior caso justamente em entradas já ordenadas, dependendo da estratégia de escolha do pivô.